Lab: Harden an Azure Databricks Workspace with Key Vault, CMK, IP Access Lists and Diagnostic Logs using Terraform

Description

In this lab, you will harden an Azure Databricks workspace with four security controls that auditors and security teams commonly require. You will build everything with Terraform, using the AzureRM (v5.6.0), AzureAD and Databricks (v1.132.0) providers.

- Secrets in Azure Key Vault → passwords and API keys live in Key Vault, not in notebooks. Databricks reads them through a Key Vault-backed secret scope.
- Customer-managed keys (CMK) → notebooks, queries and cluster disks are encrypted with your keys, which you can rotate or revoke at any time.
- IP access lists → only approved networks can reach the workspace UI and REST API.
- Diagnostic logs → workspace and Key Vault activity flows to Log Analytics for auditing and alerting.

By the end, you will prove each control works: a redacted secret in a notebook, a cluster running on encrypted disks, a request blocked by IP, and an audit trail of who read which secret.

Learning Objectives

By the end of this lab, you will be able to:

1. Create an Azure Key Vault with RBAC authorization and purge protection.
2. Create encryption keys and a secret, and grant the right identities access to them.
3. Deploy a workspace that encrypts managed services and managed disks with customer-managed keys.
4. Create a Key Vault-backed secret scope and read a secret safely from a notebook.
5. Enable IP access lists with allow and block rules.
6. Send workspace and Key Vault diagnostic logs to Log Analytics and query them with KQL.

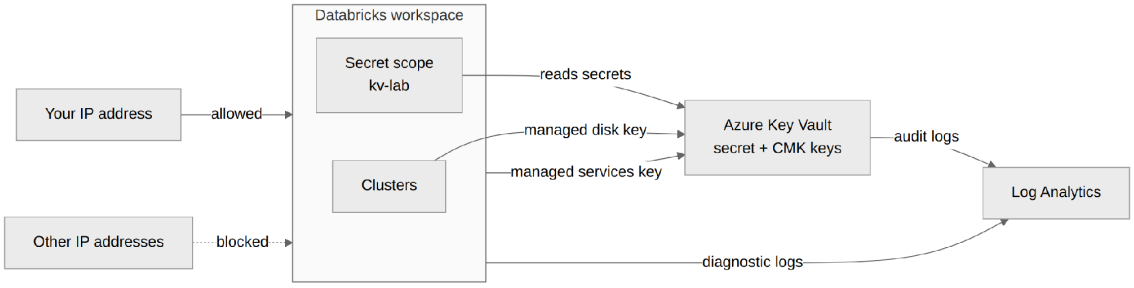
Prerequisites

Make sure you have the following before starting:

- A valid **Azure subscription** where you have **Owner**, or **Contributor** plus **User Access Administrator**. You need permission to create role assignments.
- Permission to **read service principals** in Microsoft Entra ID. Most users have this by default.
- **Terraform** and **Azure CLI** installed locally, and access to **Azure Cloud Shell** in the portal.
- Completion of the earlier lab, *Provision an Azure Databricks Workspace using Terraform*.

*CMK, IP access lists and diagnostic logs all require the **Premium** pricing tier.*

Architecture



Controls Overview

Control	Protects against	Main Terraform resources
Key Vault-backed secret scope	Credentials hard-coded in notebooks or leaked in output	azurerm_key_vault_secret, databricks_secret_scope
CMK for managed services	Notebooks, queries and secrets in the control plane readable without your key	azurerm_key_vault_key, azurerm_databricks_workspace
CMK for managed disks	Data on cluster VM disks readable without your key	azurerm_key_vault_key, azurerm_role_assignment

Control	Protects against	Main Terraform resources
IP access lists	Stolen credentials used from an unknown network	databricks_workspace_conf, databricks_ip_access_list
Diagnostic logs	Suspicious activity going unnoticed or unprovable	azurerm_monitor_diagnostic_setting

The lab has two parts, like the Unity Catalog lab. Part A builds the Azure resources. Part B configures the workspace with the Databricks provider.

Step by Step Guide

Part A: Build the Secured Azure Resources

Step 1: Authenticate and Find Your Public IP

```
az login
az account set --subscription "<your-subscription-id>"
curl -s https://ifconfig.me
```

Note your public IP. It becomes the only address allowed to reach the workspace.

Step 2: Create the Project and Configure the Providers

Create a new folder called databricks_security. Inside it, create a file named providers.tf:

```
terraform {
 required_version = ">= 1.11"

 required_providers {
  azurerm = {
   source  = "hashicorp/azurerm"
   version = "5.6.0"
  }

  azuread = {
   source  = "hashicorp/azuread"
   version = "~> 3.0"
  }
```

```
    random = {
      source  = "hashicorp/random"
      version = "~> 3.7"
    }

    time = {
      source  = "hashicorp/time"
      version = "~> 0.13"
    }
  }
}

provider "azurerm" {
  subscription_id = "<your-subscription-id>"

  features {
    key_vault {
      purge_soft_delete_on_destroy = false
    }
  }

  resource_providers_to_register = [
    "Microsoft.Databricks",
    "Microsoft.KeyVault",
    "Microsoft.OperationalInsights",
    "Microsoft.Insights",
  ]
}

provider "azuread" {}
```

Explanation:

- Terraform 1.11 or later → needed for the write-only secret value used in Step 5.
- azuread → looks up the Azure Databricks application in Microsoft Entra ID.
- time → adds a short wait while role assignments take effect.

- purge_soft_delete_on_destroy = false → the vault uses purge protection, so Terraform must not try to purge it on destroy.

Initialize the project:

```
terraform init
```

Step 3: Define Variables and Values

Create a file named variables.tf:

```
variable "resource_group_name" {
  type = string
}

variable "location" {
  type    = string
  default = "West Europe"
}

variable "workspace_name" {
  type = string
}

variable "managed_resource_group_name" {
  type = string
}

variable "my_ip_address" {
  type        = string
  description = "Your public IP in CIDR form, e.g. 203.0.113.10/32"
}

variable "tags" {
  type = map(string)
}
```

Create a file named terraform.tfvars. Replace the IP with the one from Step 1:

```
resource_group_name         = "rg-databricks-sec-lab"
location                    = "West Europe"
workspace_name              = "dbw-sec-lab"
managed_resource_group_name = "rg-databricks-sec-lab-managed"
my_ip_address               = "203.0.113.10/32"
tags = {
  environment = "dev"
  project     = "databricks-security-lab"
}
```

Step 4: Create the Resource Group and Key Vault

Create a file named key_vault.tf:

```
data "azurerm_client_config" "current" {}

resource "azurerm_resource_group" "sec" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
}

resource "random_string" "suffix" {
  length  = 6
  special = false
  upper   = false
}

resource "azurerm_key_vault" "lab" {
  name                        = "kv-dbwsec-${random_string.suffix.result}"
  location                    = azurerm_resource_group.sec.location
  resource_group_name         = azurerm_resource_group.sec.name
  tenant_id                   = data.azurerm_client_config.current.tenant_id
  sku_name                    = "standard"
  rbac_authorization_enabled = true
  purge_protection_enabled    = true
  soft_delete_retention_days = 7
```

```
  tags                = var.tags
}

resource "azurerm_role_assignment" "me_kv_admin" {
  scope              = azurerm_key_vault.lab.id
  role_definition_name = "Key Vault Administrator"
  principal_id       = data.azurerm_client_config.current.object_id
}

resource "time_sleep" "wait_for_kv_rbac" {
  create_duration = "60s"
  depends_on      = [azurerm_role_assignment.me_kv_admin]
}
```

Explanation:

- rbac_authorization_enabled = true → access is controlled with Azure role assignments instead of legacy access policies. From AzureRM v5.0, this property is required.
- purge_protection_enabled = true → a deleted vault and its keys can be recovered for 7 days and cannot be purged early. CMK requires this, so an accidental delete cannot destroy your only copy of the key.
- Key Vault Administrator → lets you create keys and secrets. Terraform runs as you.
- time_sleep → role assignments take up to a minute to take effect. Without the wait, creating keys can fail with a 403 error.

Step 5: Create the Keys and Secret, and Grant Azure Databricks Access

Create a file named keys_and_secrets.tf:

```
resource "azurerm_key_vault_key" "cmk" {
  for_each = toset(["managed-services", "managed-disks"])

  name         = "cmk-${each.key}"
  key_vault_id = azurerm_key_vault.lab.id
  key_type     = "RSA"
  key_size     = 2048
  key_opts     = ["wrapKey", "unwrapKey", "encrypt", "decrypt", "sign", "verify"]
```

```
  depends_on = [time_sleep.wait_for_kv_rbac]
}

ephemeral "random_password" "demo" {
  length  = 32
  special = false
}

resource "azurerm_key_vault_secret" "demo" {
  name             = "demo-api-key"
  key_vault_id     = azurerm_key_vault.lab.id
  value_wo         = ephemeral.random_password.demo.result
  value_wo_version = 1
  content_type     = "text/plain"

  depends_on = [time_sleep.wait_for_kv_rbac]
}

# The "AzureDatabricks" first-party application (same ID in every tenant)
data "azuread_service_principal" "azure_databricks" {
  client_id = "2ff814a6-3304-4ab8-85cb-cd0e6f879c1d"
}

resource "azurerm_role_assignment" "databricks_kv_crypto" {
  scope                = azurerm_key_vault.lab.id
  role_definition_name = "Key Vault Crypto Service Encryption User"
  principal_id         = data.azuread_service_principal.azure_databricks.object_id
}

resource "azurerm_role_assignment" "databricks_kv_secrets" {
  scope                = azurerm_key_vault.lab.id
  role_definition_name = "Key Vault Secrets User"
  principal_id         = data.azuread_service_principal.azure_databricks.object_id
}
```

Explanation:

- Two keys → one encrypts managed services, one encrypts cluster disks. Separate keys let you rotate or revoke them independently.
- ephemeral random_password + value_wo → the secret is generated and written to Key Vault without ever being stored in the Terraform state file. Increase value_wo_version to rotate it.
- AzureDatabricks application → the Microsoft-owned identity that the Databricks control plane uses. It needs Crypto Service Encryption User to use the managed services key and Secrets User to read secrets for the secret scope.

Least privilege in action: Databricks can use the keys to wrap and unwrap data and read secrets, but it cannot create, delete or export anything.

Step 6: Create the Workspace with Customer-Managed Keys

Create a file named workspace.tf:

```
resource "azurerm_databricks_workspace" "sec" {
  name                        = var.workspace_name
  resource_group_name         = azurerm_resource_group.sec.name
  location                    = azurerm_resource_group.sec.location
  sku                         = "premium"
  managed_resource_group_name = var.managed_resource_group_name

  managed_services_cmk_key_vault_key_id               =
azurerm_key_vault_key.cmk["managed-services"].id
  managed_disk_cmk_key_vault_key_id                   = azurerm_key_vault_key.cmk["managed-disks"].id
  managed_disk_cmk_rotation_to_latest_version_enabled = true

  tags = var.tags

  depends_on = [azurerm_role_assignment.databricks_kv_crypto]
}

resource "azurerm_role_assignment" "disk_encryption_set" {
  scope                = azurerm_key_vault.lab.id
  role_definition_name = "Key Vault Crypto Service Encryption User"
```

```
  principal_id        = azurerm_databricks_workspace.sec.managed_disk_identity[0].principal_id
}
```

Explanation:

- managed_services_cmk_key_vault_key_id → encrypts data Databricks stores in its control plane: notebooks, query results, secrets and job history.
- managed_disk_cmk_key_vault_key_id → encrypts the OS and data disks of every cluster VM.
- managed_disk_cmk_rotation_to_latest_version_enabled → when you rotate the key in Key Vault, disks automatically use the new version.
- managed_disk_identity → Azure creates a Disk Encryption Set in the managed resource group with its own identity. That identity must be allowed to use the disk key, or clusters will fail to start.
- depends_on → the AzureDatabricks application must have key access before the workspace is created.

Step 7: Send Diagnostic Logs to Log Analytics

Create a file named diagnostics.tf:

```
resource "azurerm_log_analytics_workspace" "sec" {
  name                = "log-dbw-sec-lab"
  location            = azurerm_resource_group.sec.location
  resource_group_name = azurerm_resource_group.sec.name
  sku                 = "PerGB2018"
  retention_in_days   = 30
  tags                = var.tags
}

data "azurerm_monitor_diagnostic_categories" "databricks" {
  resource_id = azurerm_databricks_workspace.sec.id
}

resource "azurerm_monitor_diagnostic_setting" "databricks" {
  name                           = "diag-databricks"
  target_resource_id             = azurerm_databricks_workspace.sec.id
```

```
  log_analytics_workspace_id     = azurerm_log_analytics_workspace.sec.id
  log_analytics_destination_type = "Dedicated"

  dynamic "enabled_log" {
    for_each = data.azurerm_monitor_diagnostic_categories.databricks.log_category_types

    content {
      category = enabled_log.value
    }
  }
}

resource "azurerm_monitor_diagnostic_setting" "key_vault" {
  name                       = "diag-key-vault"
  target_resource_id         = azurerm_key_vault.lab.id
  log_analytics_workspace_id = azurerm_log_analytics_workspace.sec.id

  enabled_log {
    category = "AuditEvent"
  }
}
```

Explanation:

- azurerm_monitor_diagnostic_categories → asks Azure which log categories the workspace supports, such as clusters, notebook, secrets and accounts.
- dynamic "enabled_log" → creates one enabled_log block per category, so new categories are picked up without editing the code.
- Dedicated → Databricks logs land in separate tables such as DatabricksSecrets and DatabricksClusters.
- AuditEvent → records every read of a Key Vault secret or key, including reads made by Databricks.

Step 8: Define Outputs and Deploy Part A

Create a file named outputs.tf:

```
output "databricks_workspace_url" {
  value = "https://${azurerm_databricks_workspace.sec.workspace_url}"
}

output "key_vault_name" {
  value = azurerm_key_vault.lab.name
}

output "log_analytics_workspace" {
  value = azurerm_log_analytics_workspace.sec.name
}
```

Deploy:

```
terraform fmt
terraform validate
terraform plan
terraform apply --auto-approve
```

Part A takes about 10 minutes.

Part B: Configure the Workspace Controls

Step 9: Add the Databricks Provider

In providers.tf, add the Databricks provider inside the required_providers block:

```
    databricks = {
      source  = "databricks/databricks"
      version = "1.132.0"
    }
```

Then add this block at the end of the file:

```
provider "databricks" {
  host                        = azurerm_databricks_workspace.sec.workspace_url
  azure_workspace_resource_id = azurerm_databricks_workspace.sec.id
}
```

Download the provider:

```
terraform init -upgrade
```

Step 10: Create the Key Vault-Backed Secret Scope

Create a file named secret_scope.tf:

```
resource "databricks_secret_scope" "kv" {
  name = "kv-lab"

  keyvault_metadata {
    resource_id = azurerm_key_vault.lab.id
    dns_name    = azurerm_key_vault.lab.vault_uri
  }
}
```

Explanation:

- Key Vault-backed scope → Databricks does not store a copy of the secrets. Each read goes to Key Vault through the AzureDatabricks application you granted Key Vault Secrets User in Step 5.
- Read-only from Databricks → secrets are created, rotated and deleted in Key Vault only. Databricks users can read them but cannot change them.
- Created by a user → Azure only allows a Key Vault-backed scope to be created by a Microsoft Entra ID user, not a service principal. Terraform runs as you, so this works.

Step 11: Enable IP Access Lists

Create a file named ip_access.tf:

```
resource "databricks_workspace_conf" "security" {
  custom_config = {
    "enableIpAccessLists" = "true"
  }
}

resource "databricks_ip_access_list" "allow" {
  label        = "allow-my-ip"
  list_type    = "ALLOW"
  ip_addresses = [var.my_ip_address]

  depends_on = [databricks_workspace_conf.security]
```

```
}


resource "databricks_ip_access_list" "block" {
  label       = "block-known-bad-range"
  list_type   = "BLOCK"
  ip_addresses = ["198.51.100.0/24"]

  depends_on = [databricks_workspace_conf.security]
}
```

Explanation:

- enableIpAccessLists → turns the feature on for the workspace.
- ALLOW list → once an allow list exists, only these addresses can reach the workspace UI and REST API.
- BLOCK list → checked first. An address on a block list is refused even if it is also in an allow range. In production, this holds ranges flagged by your security team.
- Scope → IP access lists protect the UI and API. They do not affect traffic from clusters to the control plane.

Your own IP must be in the allow list, because Terraform also calls the workspace API. If you remove it, you lock yourself and Terraform out.

Step 12: Add an Output and Deploy Part B

Add this to outputs.tf:

```
output "secret_scope_name" {
  value = databricks_secret_scope.kv.name
}
```

Deploy:

```
terraform fmt
terraform validate
terraform plan
terraform apply --auto-approve
```

IP access list changes take a few minutes to take effect.

Step 13: Verify Each Control

13.1 Customer-managed keys

1. In the Azure Portal, open **dbw-sec-lab** and go to **Settings → Encryption**. Both **managed services** and **managed disks** show your Key Vault keys.
2. Open the managed resource group **rg-databricks-sec-lab-managed**. It contains a **Disk Encryption Set** that uses **cmk-managed-disks**.

13.2 Start a cluster on encrypted disks

1. Open the **databricks_workspace_url** and sign in.
2. Go to **Compute → Create compute**. Choose **Single node**, the latest **LTS** runtime and the smallest node type. Set **Terminate after** to **20 minutes**.
3. Wait until the cluster is **Running**. In the managed resource group, open one of the new cluster disks and check **Encryption**. It shows a **customer-managed key**.

13.3 Read a secret from the Key Vault-backed scope

Create a Python notebook attached to the cluster and run:

```
print(dbutils.secrets.listScopes())
print(dbutils.secrets.list("kv-lab"))

api_key = dbutils.secrets.get(scope="kv-lab", key="demo-api-key")
print(api_key)
print(len(api_key))
```

Command	Expected output	What it shows
listScopes	A scope named kv-lab	The scope was created by Terraform
list	A key named demo-api-key	Databricks can see secrets in Key Vault
print(api_key)	[REDACTED]	Databricks hides secret values in notebook output

Command	Expected output	What it shows
len(api_key)	32	The real value is available to code, just not displayed

13.4 Test the IP access list

Run this from your laptop terminal, replacing the URL with your databricks_workspace_url:

```
curl -s -o /dev/null -w "%{http_code}\n"
https://adb-1234567890.12.azuredatabricks.net/api/2.0/clusters/list
```

Now open Azure Cloud Shell in the portal, which runs from a different Microsoft IP address, and run the same command.

Where you run it	Expected code	Meaning
Your laptop	401	The request reached the workspace and was rejected only because no token was sent
Azure Cloud Shell	403	The request was blocked by the IP access list before authentication

13.5 Query the audit logs

In the Azure Portal, open log-dbw-sec-lab → Logs and run:

```
DatabricksSecrets
| where TimeGenerated > ago(2h)
| project TimeGenerated, ActionName, Identity, RequestParams
| order by TimeGenerated desc
```

You see getSecret and listSecrets entries with your identity and the kv-lab scope.

```
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.KEYVAULT" and OperationName == "SecretGet"
| project TimeGenerated, OperationName, ResultType, CallerIPAddress
| order by TimeGenerated desc
```

You see the same secret reads from the Key Vault side, made by Azure Databricks on your behalf.

```
DatabricksClusters
| where TimeGenerated > ago(2h)
| project TimeGenerated, ActionName, Identity
| order by TimeGenerated desc
```

Diagnostic logs can take 10 to 15 minutes to appear.

Step 14: Clean Up Resources

Terminate the cluster in the Databricks UI. Then, from the same network so your IP is still allowed, run:

```
terraform destroy --auto-approve
```

Because of purge protection, the Key Vault stays in a soft-deleted state for 7 days before Azure removes it permanently. This is expected. It is the safety net that stops an attacker, or a mistake, from destroying your encryption keys.

Troubleshooting

- 403 Forbidden when creating keys or the secret → your Key Vault Administrator role was not active yet. Run terraform apply again.
- Error about ephemeral resources or value_wo → upgrade Terraform to 1.11 or later.
- Workspace creation fails with a key access error → check the databricks_kv_crypto role assignment and that purge protection is enabled on the vault.
- Cluster fails to start with a disk encryption error → the disk_encryption_set role assignment may still be propagating. Wait five minutes and start the cluster again.
- Secret scope creation fails with a token or service principal error → Key Vault-backed scopes must be created by a user. Run Terraform with your own az login, not a service principal.
- Locked out after your IP changed → reconnect from the original network if you can. Otherwise, remove the Databricks resources from state with terraform state rm databricks_ip_access_list.allow databricks_ip_access_list.block databricks_workspace_conf.security databricks_secret_scope.kv, then run terraform

destroy. The workspace itself is deleted through Azure, which the IP access list does not block.
- No rows in Log Analytics → wait 15 minutes, and confirm the diag-databricks and diag-key-vault settings exist.

Outcomes

You've hardened an Azure Databricks workspace with four layered security controls, all defined in Terraform. You now know how to:

- Build an Azure Key Vault with RBAC, purge protection and least-privilege role assignments.
- Write a secret with a write-only value so it never appears in the Terraform state file.
- Encrypt managed services and managed disks with customer-managed keys, including automatic key version rotation for disks.
- Expose Key Vault secrets to notebooks through a Key Vault-backed secret scope, with values redacted in output.
- Restrict the workspace UI and API with IP access lists, and recover safely from a lockout.
- Send Databricks and Key Vault audit logs to Log Analytics and trace every secret read with KQL.

This sets the foundation for CMK on the DBFS root storage, automated key rotation policies, and alerting rules that notify your security team when secrets are read outside working hours.